

Regresión Lineal

La recta que la máxima verosimilitud escoge

Enrique Bolaños Abner Xocop

18 de agosto de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** Predecir un número
- 2.** Mínimos cuadrados es MLE
- 3.** Cómo resolverlo
- 4.** Evaluar honestamente
- 5.** Ridge y regularización
- 6.** Sesgo y varianza
- 7.** Ejercicios
- 8.** Cierre

Del clasificador al regresor

El jueves clasificamos: la salida era una **etiqueta**. Hoy la salida es un **número**.

- Precio de una casa en la zona 15, dados metros y habitaciones
- Demanda de energía de mañana, dado el clima y el día
- Nota final de un estudiante, dadas sus notas parciales

La regresión lineal es el modelo más viejo y más usado del oficio. Vamos a llegar a ella **derivándola**, no memorizando la fórmula.

Hoy se entrega la **HDT 2** y se asigna la **HDT 3**.

El modelo lineal

Con n observaciones y d features:

$$y = Xw + \varepsilon, \quad X \in \mathbb{R}^{n \times d}, \quad w \in \mathbb{R}^d$$

- Cada peso w_j es **cuánto cambia y si x_j sube en una unidad**, dejando lo demás fijo
- ε es lo que el modelo no explica: ruido, variables faltantes, error de medición

¿Y el intercepto? Se obtiene gratis agregando una **columna de unos** a X : su peso es el valor esperado de y cuando todas las features valen cero.

Lineal en los pesos, no necesariamente en los datos

La pérdida: suma de errores al cuadrado

Para un candidato w , el residuo de la observación i es $y_i - x_i^\top w$. Sumamos los residuos al cuadrado:

$$\mathcal{L}(w) = \sum_{i=1}^n \left(y_i - x_i^\top w \right)^2 = \|y - Xw\|^2$$

¿Por qué al cuadrado y no en valor absoluto? Dos razones cómodas: es diferenciable en todas partes y tiene solución cerrada.

Y una tercera razón, mucho mejor, en la siguiente diapositiva.

Geométricamente: Xw recorre el subespacio generado por las columnas de X , y minimizar $\mathcal{L}$ es **proyectar** y sobre ese subespacio.

El momento “aha”: de dónde sale el cuadrado

Supongamos que el ruido es normal e independiente: $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. Entonces cada observación tiene densidad

$$p(y_i | x_i, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^\top w)^2}{2\sigma^2}\right)$$

Tomando log-verosimilitud de las n observaciones independientes:

$$\ell(w) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - x_i^\top w)^2$$

El primer término no depende de w y el segundo lleva signo negativo, así que

$$\arg \max_w \ell(w) = \arg \min_w \|y - Xw\|^2$$

MÁS ALLÁ DEL CÓDIGO

Toda pérdida es un supuesto probabilístico disfrazado.

Ecuaciones normales

Derivamos $\mathcal{L}(w) = \|y - Xw\|^2$ respecto a w y la igualamos a cero:

$$\nabla_w \mathcal{L}(w) = -2X^\top (y - Xw) = 0$$

$$X^\top X w = X^\top y$$

$$\hat{w} = (X^\top X)^{-1} X^\top y$$

Un problema de optimización que se resuelve **sin iterar**: pura álgebra lineal.

Nunca inviertan la matriz

`np.linalg.inv` es más lento y numéricamente peor que resolver el sistema directo. Usen `np.linalg.solve`.

OLS en tres líneas

```
1 rng = np.random.default_rng(42)
2 n = 200
3 X = rng.normal(size=(n, 3))
4 w_real = np.array([2.0, -1.0, 0.5])
5 y = X @ w_real + 4.0 + rng.normal(0, 0.5, n)
6
7 Xb = np.hstack([np.ones((n, 1)), X]) # intercepto
8 w = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)
9 w # ~ [4.0, 2.0, -1.0, 0.5]
```

Datos simulados a propósito: conocemos la respuesta correcta, así que podemos verificar que el estimador la recupera.

Descenso de gradiente: la otra ruta

Si $X^T X$ es enorme (millones de features), la solución cerrada no cabe. Bajamos por la pendiente:

```
1 w = np.zeros(Xb.shape[1])
2 lr = 0.05
3 for _ in range(500):
4     grad = -2 / n * Xb.T @ (y - Xb @ w)
5     w -= lr * grad
```

Mismo destino, camino distinto. Aquí es opcional; en deep learning (semanas 13-15) va a ser la **única** opción.

Si `lr` es muy grande diverge; si es muy chico no llega. Prueben.

R^2 : qué fracción de la varianza explicamos

```
1 pred = Xb @ w
2 resid = y - pred
3
4 ss_res = (resid ** 2).sum()
5 ss_tot = ((y - y.mean()) ** 2).sum()
6 r2 = 1 - ss_res / ss_tot
```

- $R^2 = 1$: predicción perfecta; $R^2 = 0$: igual que predecir la media
- Puede ser **negativo** en datos de prueba (peor que la media)
- Siempre sube al agregar features, aunque sean ruido: por eso nunca se juzga con R^2 de entrenamiento

Miren los residuos, no solo el número

Grafiquen residuos contra predicciones. Lo que deben ver: una **nube sin forma** alrededor de cero. Lo que delata problemas:

- Una **curva**: la relación no era lineal, falta un término
- Un **embudo** que se abre: la varianza no es constante
- Puntos **muy lejos**: outliers, que el cuadrado castiga con fuerza

Extrapolar es el pecado clásico

Un modelo ajustado con casas de 80 a 200 m² no sabe nada de una de 600 m². La recta se extiende felizmente hasta el infinito; los datos no.

Cuando $X^T X$ se porta mal

La fórmula cerrada supone que $X^T X$ es invertible. En la práctica se degrada mucho antes de dejar de serlo:

- **Colinealidad:** dos features casi redundantes (metros cuadrados y metros construidos)
- Más features que observaciones ($d > n$): infinitas soluciones exactas
- **Overfitting:** con muchas features el modelo memoriza el ruido

El síntoma es siempre el mismo: pesos **enormes** y de signo arbitrario, que se cancelan entre sí y cambian por completo si mueven un dato.

Pesos gigantes son señal de un modelo inestable

Ridge: penalizar los pesos grandes

Agregamos a la pérdida un castigo por magnitud:

$$\mathcal{L}_{\text{ridge}}(w) = \|y - Xw\|^2 + \lambda\|w\|^2$$

La solución cerrada casi no cambia:

$$\hat{w}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$$

- Sumar λI vuelve invertible a cualquier matriz
- $\lambda = 0$ recupera OLS; $\lambda \rightarrow \infty$ manda todos los pesos a cero
- Estandaricen las features antes: la penalización compara magnitudes
- Nunca penalicen el intercepto: no es una pendiente
- λ se elige con validación, nunca con el error de entrenamiento

Ridge es MAP con prior normal

Pongamos un prior sobre los pesos: $w \sim \mathcal{N}(0, \tau^2 I)$, es decir “creo que los pesos son chicos”. El posterior cumple

$$\log p(w \mid X, y) = -\frac{1}{2\sigma^2} \|y - Xw\|^2 - \frac{1}{2\tau^2} \|w\|^2 + C$$

Maximizar el posterior (MAP) es entonces minimizar

$$\|y - Xw\|^2 + \frac{\sigma^2}{\tau^2} \|w\|^2, \quad \lambda = \frac{\sigma^2}{\tau^2}$$

Un prior más apretado (menor τ) es un λ mayor. La regularización no es un truco: es una creencia previa, escrita como penalización.

La descomposición del error

El error esperado de predicción en un punto se parte en tres:

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \underbrace{\text{sesgo}^2}_{\text{modelo muy rígido}} + \underbrace{\text{varianza}}_{\text{modelo muy sensible}} + \underbrace{\sigma^2}_{\text{ruido irreducible}}$$

- **Sesgo:** el error que cometemos por asumir una forma equivocada
- **Varianza:** cuánto cambia el modelo si cambiamos la muestra
- **Ruido:** lo que nadie puede predecir, ni con datos infinitos

Más complejidad baja el sesgo y sube la varianza; el mínimo del error total está **en el medio**. Subir λ introduce sesgo a propósito para comprar una reducción mayor de varianza: eso es mover el punto en esa curva.

A mano y con scikit-learn

```
1 def ridge(Xb, y, lam):
2     P = lam * np.eye(Xb.shape[1])
3     P[0, 0] = 0.0          # no penalizar intercepto
4     return np.linalg.solve(Xb.T @ Xb + P, Xb.T @ y)
5
6 from sklearn.linear_model import LinearRegression, Ridge
7 ols = LinearRegression().fit(X, y)
8 rdg = Ridge(alpha=1.0).fit(X, y)
9 ols.coef_, ols.intercept_, rdg.coef_
```

sklearn agrega la columna de unos por ustedes (`fit_intercept=True`) y tampoco penaliza el intercepto. Su α es nuestro λ , y sus coeficientes coinciden con los suyos.

Ejercicio 1: OLS a mano vs sklearn

```
1 rng = np.random.default_rng(0)
2 n = 300
3 X = rng.normal(size=(n, 4))
4 w_real = np.array([1.5, -2.0, 0.0, 3.0])
5 y = X @ w_real - 1.0 + rng.normal(0, 1.0, n)
6
7 def ols(X, y):
8     # ¿Que va aqui?
9     # 1) agregar columna de unos
10    # 2) resolver con np.linalg.solve (NO inv)
11    pass
12
13 # Verificacion: comparar con LinearRegression
```

Ejercicio 2: un feature de puro ruido

```
1 ruido = rng.normal(size=(n, 1))
2 X2 = np.hstack([X, ruido])    # feature sin señal
3
4 # ¿Que va aquí?
5 # a) ajustar OLS sobre X2 y ver el peso del ruido
6 # b) ajustar Ridge (lam=10) y ver ese mismo peso
7 # c) repetir con otra semilla: ¿cual peso se mueve mas?
8
9 # El peso ideal seria exactamente 0
```

La pregunta de fondo: ¿cuál de los dos estimadores es más honesto sobre lo que no sabe?

Ejercicio 3: la curva de λ

```
1 lams = np.logspace(-3, 3, 25)
2 err_train, err_test = [], []
3
4 for lam in lams:
5     # ¿Que va aqui?
6     # 1) ajustar ridge con datos de train
7     # 2) guardar MSE de train y MSE de test
8     pass
9
10 # plt.semilogx(lams, err_train, lams, err_test)
```

¿Dónde está el mínimo de la curva de test? Eso es sesgo-varianza, visto de frente.

Lo esencial de hoy

- El modelo lineal $y = Xw + \varepsilon$; el intercepto es una columna de unos
- **Mínimos cuadrados** = **MLE** con ruido normal: el cuadrado no es arbitrario
- **Ecuaciones normales**: $X^\top X w = X^\top y$, con solve, nunca inv
- Evaluar con R^2 y residuos; extrapolar es el pecado clásico
- **Ridge** = **MAP** con prior normal sobre los pesos; λ es la perilla
- Error = sesgo² + varianza + ruido: el óptimo está en el medio

Próxima clase (jueves 20): EDA y Visualización

Se asigna la **HDT 3** (entrega: martes 25 de agosto)